R: limpieza, manipulación y exploración de datos

2do encuentro: Estructuras de control y datos

Jonatán Perren

24 de marzo de 2026

Factores (Factor)

Los factores son un tipo de dato especial para variables categóricas (como «Nivel educativo», «Sexo», «Provincia»). Aunque parecen texto, internamente R los guarda como números enteros asociados a etiquetas.

¿Por qué usarlos?

Son útiles para modelos estadísticos y para ordenar gráficos de forma no alfabética (por ejemplo, «Bajo», «Medio», «Alto»). También son más eficientes a la hora de guardar grandes bases de datos.

Operación	Función	Ejemplo de Uso	Resultado
Crear factor	`factor()`	`factor(c("A", "B", "A"))`	Niveles: A, B
Ordenado	`factor()`	`factor(c("B", "A"), ordered = T)`	A B
Ver niveles	`levels()`	`levels(mi_factor)`	Nombres de cat
Contar	`nlevels()`	`nlevels(mi_factor)`	Cantidad de niv
Frecuencias	`table()`	`table(mi_factor)`	Conteo por cate
Renombrar	`levels()`	`levels(f) <- c("N1", "N2")`	Niveles cambiad
A numérico	`as.numeric()`	`as.numeric(f)`	Códigos interno
A carácter	`as.character()`	`as.character(f)`	Vector de texto

```r
# Crear un factor básico
sexo <- factor(c("Masculino", "Femenino", "Femenino", "Masculino"))

sexo
```

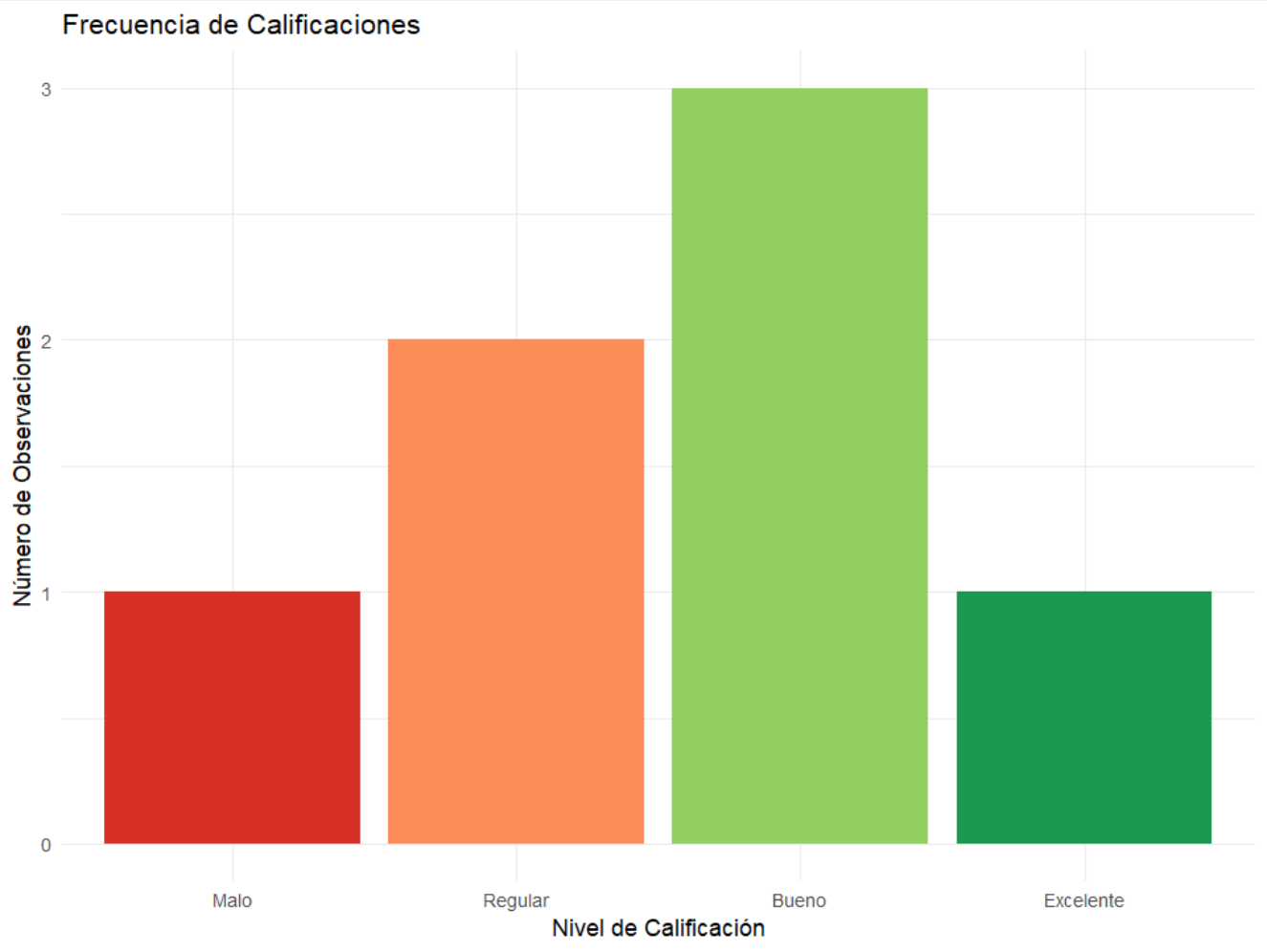
```
[1] Masculino Femenino  Femenino  Masculino
Levels: Femenino Masculino
```

```r
levels(sexo)
```

```
[1] "Femenino"  "Masculino"
```

El siguiente ejemplo contiene funciones que aun no vimos. Pero es traído solo para mostrar un caso de uso de los factores.

```r
library(tidyverse)

# Datos en bruto obtenidos
datos <- c("Bueno", "Malo", "Excelente", "Regular", "Bueno", "Regular", "Bueno")

# Formalización de la variable
calificacion <- factor(datos,
  levels = c("Malo", "Regular", "Bueno", "Excelente"), #Niveles en orden
  ordered = TRUE #Y que se respete el orden
)

# Crear un data frame con el resultado de table(calificacion)
df_calif <- as_tibble(table(calificacion))

# Cambiamos el nombre de las columnas
colnames(df_calif) <- c("Calificacion", "Frecuencia")

# La gráfica (más adelante veremos qué es cada cosa)
ggplot(df_calif, aes(x = Calificacion, y = Frecuencia, fill = Calificacion)) +
  geom_bar(stat = "identity") +
  scale_fill_manual(
    values = c(
    "Malo"      = "#D73027",
    "Regular"   = "#FC8D59",
    "Bueno"     = "#91CF60",
    "Excelente" = "#1A9850")
  ) +
  labs(
    title = "Frecuencia de Calificaciones",
    x     = "Nivel de Calificación",
    y     = "Número de Observaciones"
  ) +
  theme_minimal() +
  theme(legend.position = "none")
```


Frecuencia de Calificaciones
Número de Observaciones
3
2
1
0
Malo
Regular
Bueno
Excelente
Nivel de Calificación

Manipulación avanzada con forcats

El paquete `forcats` (parte del ecosistema tidyverse) ofrece herramientas poderosas para resolver problemas comunes con factores. Podés encontrar el cheatsheet en el aula virtual o en **este enlace**.

Función	Descripción
`fct_reorder()`	Reordena niveles según otra variable (útil para gráficos).
`fct_relevel()`	Cambia manualmente el orden de los niveles.
`fct_lump()`	Agrupa los niveles menos frecuentes en «Otros».
`fct_infreq()`	Reordena niveles por frecuencia de aparición.

```r
# install.packages("forcats")
library(forcats)

# Ejemplo: Reordenar por frecuencia
letras <- factor(c("a", "b", "b", "c", "c", "c"))
levels(letras) # a, b, c (alfabético)

letras_freq <- fct_infreq(letras)
levels(letras_freq) # c, b, a (por frecuencia)

# Ejemplo: Agrupar niveles poco comunes
paises <- factor(c("Arg", "Bra", "Arg", "Chi", "Ury", "Bra", "Arg", "Per"))
fct_lump(paises, n = 2) # Mantiene los 2 más comunes, el resto es "Other"
```

Estructuras de datos compuestas

Hasta ahora hemos trabajado con vectores y matrices, que tienen una restricción fuerte: todos sus elementos deben ser del mismo tipo (todos lógicos, todos números o todos texto). Sin embargo, en la vida real, los conjuntos de datos suelen venir mezclados. Por ejemplo, en una encuesta se puede preguntar por el sexo (categórico/factor), por la edad (numérico), o por si tiene trabajo formal (lógico).

Para manejar esto, R nos ofrece las listas y los data frames.

Listas

Las listas son objetos muy flexibles (y esto no necesariamente es una ventaja) que pueden contener cualquier cosa: vectores, matrices, otras listas, funciones, etc. No requieren que sus elementos sean del mismo tipo **ni de la misma longitud**.

```r
mi_lista <- list(
  nombre = "Lucía",
  edad = 30,
  notas = c(8, 9, 10),
  aprobado = TRUE
)

mi_lista
```

```
$nombre
[1] "Lucía"

$edad
[1] 30

$notas
[1]  8  9 10

$aprobado
[1] TRUE
```

Acceso a elementos

A diferencia de los vectores, las listas tienen dos tipos de corchetes:

- `[]` (corchete simple): Devuelve una sub-lista. **Importante!** El resultado sigue siendo una lista.
- `[[]]` (doble corchete): Extrae el contenido real del elemento.
- `$` (signo peso): Accede a los elementos por nombre (si lo tienen). Extrae como `[[]]`.

```
mi_lista[1]       # Devuelve una lista con el elemento "nombre"
typeof(mi_lista[1])
```

```
[1] "list"
```

```
mi_lista[[1]]     # Devuelve el vector de caracter "Lucía"
typeof(mi_lista[[1]])
```

```
[1] "character"
```

```
mi_lista$notas    # Devuelve el vector de notas
mi_lista$notas[2] # El segundo elemento del vector notas
```

```
[1] 9
```

Data frames

El data frame es, sin lugar a dudas, **la estructura más importante** para el análisis de datos en R. Si venís del mundo de Excel, podés pensarlo como una hoja de cálculo, donde cada fila es una observación y cada columna es una variable.

Es la estructura fundamental porque agrupa vectores de diferentes tipos (caracteres, números, lógicos) en un mismo objeto rectangular, manteniendo la integridad de los datos:

1. Rectangularidad: Todas las columnas deben tener la misma longitud.
2. Heterogeneidad de columnas: La columna «Edad» puede ser numérica mientras que «Nombre» es texto.
3. Homogeneidad interna: Dentro de una misma columna, todos los datos deben ser del mismo tipo.

Técnicamente, en R un data frame no es más que una lista de vectores de igual longitud, que se visualiza como una tabla. Esta dualidad (lista/tabla) es lo que lo hace tan potente.

```r
# Creación manual de un data frame
datos_alumnos <- data.frame(
  nombre = c("Ana", "Pedro", "Maria"),
  edad = c(22, 25, 20),
  nota = c(8.5, 6.0, 9.5),
  beca = c(TRUE, FALSE, TRUE)
)

datos_alumnos
```

```
  nombre edad nota  beca
1    Ana   22  8.5  TRUE
2  Pedro   25  6.0 FALSE
3  Maria   20  9.5  TRUE
```

- `nrow()`, `ncol()`, `dim()`: Dimensiones.
- `names()` o `colnames()`: Nombres de las columnas.
- `summary()`: Resumen estadístico rápido.
- `str()`: Estructura interna (muy útil para verificar tipos de datos).

```r
str(datos_alumnos)
```

```
'data.frame': 3 obs. of  4 variables:
$ nombre: chr  "Ana" "Pedro" "Maria"
$ edad  : num  22 25 20
$ nota  : num  8.5 6 9.5
$ beca  : logi  TRUE FALSE TRUE
```

```r
summary(datos_alumnos)
```

```
    nombre               edad            nota         beca
Length:3           Min.   :20.00   Min.   :6.00   Mode :logical
Class :character   1st Qu.:21.00   1st Qu.:7.25   FALSE:1
Mode  :character   Median :22.00   Median :8.50   TRUE :2
                   Mean   :22.33   Mean   :8.00
                   3rd Qu.:23.50   3rd Qu.:9.00
                   Max.   :25.00   Max.   :9.50
```

Acceso a datos (indexación)

Podemos tratarlo como matriz `[fila, columna]` o como lista `$`columna.

```r
datos_alumnos$nombre       # Columna 'nombre' como vector
```

```
[1] "Ana"   "Pedro" "Maria"
```

```r
datos_alumnos[1, ]         # Primera fila
```

```
  nombre edad nota beca
1    Ana   22  8.5 TRUE
```

```r
datos_alumnos[, "edad"]    # Columna 'edad'
```

```
[1] 22 25 20
```

```r
datos_alumnos[datos_alumnos$nota > 8, ] # Filtrar alumnos con nota > 8
```

```
  nombre edad nota beca
1    Ana   22  8.5 TRUE
3  Maria   20  9.5 TRUE
```

Tibbles

Si bien `data.frame` es la base, en el ecosistema tidyverse se utiliza una versión modernizada llamada tibble.

Un `tibble` es un data frame que hace menos cosas (no convierte texto a factores automáticamente, no cambia nombres de variables) y se muestra mejor en consola (dice qué tipo de dato es cada columna y no imprime todo si la tabla es gigante).

```r
#install.packages("tibble")
library(tibble)

# Convertir un data frame a tibble
as_tibble(datos_alumnos)
```

```
# A tibble: 3 × 4
  nombre  edad  nota beca
  <chr>  <dbl> <dbl> <lgl>
1 Ana       22   8.5 TRUE
2 Pedro     25   6   FALSE
3 Maria     20   9.5 TRUE
```

Para crear una tabla por filas podemos utilizar `tribble()`. Es útil para tablas pequeñas que tengan que ser cargadas manualmente.

```r
  # Crear un tibble por filas con tribble
tabla_manual <- tribble(
~nombre, ~edad, ~nota, ~beca,
"Ana"  ,    22,   8.5,  TRUE,
"Pedro",    25,     6, FALSE,
"María",    20,   9.5,  TRUE
)

tabla_manual
```

```
# A tibble: 3 × 4
  nombre  edad  nota beca
  <chr>  <dbl> <dbl> <lgl>
1 Ana       22   8.5 TRUE
2 Pedro     25   6   FALSE
3 María     20   9.5 TRUE
```

```r
# Obtener lista de estudiantes con acceso a beca
# Funciona como cualquier data frame
tabla_manual[tabla_manual$beca == TRUE, ]
```

```
# A tibble: 2 × 4
  nombre  edad  nota beca
  <chr>  <dbl> <dbl> <lgl>
1 Ana       22   8.5 TRUE
2 María     20   9.5 TRUE
```

Estructuras de control de flujo

El control de flujo nos permite decidir qué código ejecutar y cuántas veces. Sin embargo, es importante hacer una distinción crucial en R:

A diferencia de lenguajes como Julia o Python, donde los bucles (`for`, `while`) son el pan de cada día, en R intentamos evitarlos siempre que sea posible.

La filosofía de R es vectorial: en lugar de iterar elemento por elemento, aplicamos operaciones a todo el vector simultáneamente. Esto no solo hace el código más legible, sino mucho más eficiente. Aun así, estas estructuras existen y son fundamentales para cuando la vectorización no es posible o para tareas de programación más general (como por ejemplo, implementar simulaciones complejas).

if y else

Permiten ejecutar un bloque de código solo si se cumple una condición lógica (TRUE).

`if` solo evalúa un único valor lógico, no un vector (para vectores usaremos `ifelse`).

```
if (condicion) {
  # Instrucción si es VERDADERO
} else {
  # Instrucción si es FALSO
}
```

```
promedio ← 5.5

if (promedio ≥ 6) {
  print("Aprobado")
} else {
  print("Desaprobado")
}
```

```
[1] "Desaprobado"
```

Se pueden encadenar condiciones con `else if`:

```
nota ← 8

if (nota == 10) {
  mensaje ← "Excelente"
} else if (nota ≥ 8) {
  mensaje ← "Muy bueno"
} else if (nota ≥ 6) {
  mensaje ← "Bueno"
} else {
  mensaje ← "Insuficiente"
}
print(mensaje)
```

```
[1] "Muy bueno"
```

Condiciones vectorizadas

Como `if` no funciona bien con vectores largos, R tiene una función optimizada llamada `ifelse()`. Esta función toma un vector de condiciones y devuelve un vector de resultados del mismo largo.

```
ifelse(test, si_verdadero, si_falso)
```

```
notas_curso <- c(4, 9, 6, 2, 10)
estado <- ifelse(notas_curso ≥ 6, "Aprobado", "Desaprobado")
estado
```

```
[1] "Desaprobado" "Aprobado"    "Aprobado"    "Desaprobado" "Aprobado"
```

Esto es equivalente a escribir un bucle `for` que recorra cada nota, pero es mucho más rápido y se escribe en una sola línea. Aunque aun no hemos visto el bucle `for`, adjunto un ejemplo de cómo debería implementarse lo mismo utilizando esta aproximación, solo para comparar las extensiones.

```
notas_curso ← c(4, 9, 6, 2, 10)
estado ← logical(length(notas_curso))
for (indice in 1:length(notas_curso)){
  estado[indice] ← notas_curso[indice] ≥ 6
}

estado
```

```
[1] FALSE  TRUE  TRUE FALSE  TRUE
```

¿Qué hace `logical()`? ¿Qué recibe como argumento y qué devuelve?

Para casos con múltiples categorías, existe `dplyr::case_match()`, que veremos más adelante, aunque se pueden anidar `ifelse()` (poco legible):

```r
# Calificaciones según régimen UNL.
clasificacion <- ifelse(
  notas_curso < 6, "Insuficiente", ifelse(
    notas_curso < 7, "Aprobado", ifelse(
      notas_curso < 8, "Bueno", ifelse(
        notas_curso < 9, "Muy bueno", ifelse(
          notas_curso < 10, "Distinguido", "Sobresaliente"
  ))))) # Poco legible

clasificacion
```

```
[1] "Insuficiente"  "Distinguido"   "Aprobado"      "Insuficiente"  "Sobresaliente"
```

Bucles for

Los bucles `for` permiten repetir un bloque de código una cantidad determinada de veces. Se suele iterar sobre una secuencia numérica o sobre los elementos de un vector.

```
for (variable in secuencia) {
  # Código a repetir
}
```

```
animales <- c("Gato", "Perro", "Pájaro")

for (animal in animales) {
  print(paste("El", animal, "es una mascota"))
}
```

```
[1] "El Gato es una mascota"
[1] "El Perro es una mascota"
[1] "El Pájaro es una mascota"
```

```r
valores <- c(2, 4, 6)
cuadrados <- numeric(length(valores)) # Pre-asignar espacio es buena práctica

for (i in 1:length(valores)) {
  cuadrados[i] <- valores[i]^2
}
cuadrados
```

```
[1]  4 16 36
```

¿Qué hace `numeric()`? Comparelo con `logical()`.

En R, siempre intenta usar operaciones vectorizadas antes de escribir un bucle `for`. Los bucles son útiles para tareas recursivas (donde el paso actual depende del anterior) o procesos complejos que no son puramente matemáticos (como leer múltiples archivos), pero para cálculos directos son menos eficientes y más verbosos.

En este caso, es más eficiente hacer

```r
valores <- c(2, 4, 6)

cuadrados <- valores^2

cuadrados
```

```
[1]  4 16 36
```

Como dije antes, a menos que estés construyendo una simulación compleja, no va a ser necesario que recurras a cosas como un `for`.

Ejercicios

1. Creá un factor llamado `color` con los niveles «Rojo» , «Verde» y «Azul». ¿Qué pasa si intentás agregar un cuarto color «Amarillo» directamente a uno de los elementos? Intentá más de una variante: con `c()` o con `append()` (y si no sabés que hace `append`, entonces `?append`).
2. Tenés datos de talles de remeras en una tienda:

```
c("M", "M", "S", "M", "M", "S", "S", "M", "M", "S", "S", "L",
"M", "S", "L", "S", "S", "M", "M", "S", "M", "M", "L", "S",
"L", "M", "L", "L", "M", "M", "M", "S", "L", "M", "S", "S",
"M", "S", "S", "M", "S", "M", "M", "S", "L", "L", "S", "S",
"M", "S")
```

   Definí un factor `talle` con niveles en el orden correcto. ¿Por qué el orden importa en este contexto?
3. Del factor `talle` anterior, ¿cuántas remeras hay de cada talle? ¿Cuál es el talle más vendido? (`?table` 😉)
4. En el factor `color`, querés que «Azul» aparezca primero en los gráficos en lugar del orden alfabético. ¿Cómo reorganizarías los niveles?
5. Si convertís el factor `talle` a números (con `as.numeric()`), ¿qué representan esos números? ¿Por qué crees que R lo hace así internamente?

6. Imaginá que estás registrando información de un único estudiante. Creá una estructura que almacene su nombre, edad y un vector con sus últimas 10 notas de exámenes.
   - ¿Qué tipo de dato usarías para cada columna?
   - ¿Cómo calcularías el promedio académico del estudiante?
   - El estudiante acaba de inscribirse en un nuevo curso. ¿Cómo agregarías esta información (`cursando = TRUE`) a su registro sin perder los datos anteriores?
   - Agrega un elemento llamado `evaluacion_aprobada` y agregalo al estudiante. Esta será un vector con `TRUE` si el estudiante aprobó la i-ésima evaluación o `FALSE` en caso contrario.
7. Una empresa tiene información sobre 3 empleados y sus salarios en dólares: Ana (25 años, $1000), Bob (30 años, $1200) y Sol (28 años, $1100). Estructura estos tres datos en formato tabular.
   - Inspeccioná la estructura interna de la tabla `empleados`. ¿R detectó correctamente que nombres son texto y edades/sueldos son números?
   - ¿Cuánto cobra cada empleado? Mostrá solo la columna de sueldos.
   - Necesitás el nombre del empleado que está en la segunda fila. ¿Cómo lo extraés? (pensá en dos soluciones diferentes)
   - Convertí el data frame `empleados` al formato moderno de `tibble`. ¿Notás alguna diferencia en cómo se visualiza?
   - Intentá responder a estas preguntas filtrando y operando vectorialmente: ¿Cuánto dinero reciben los tres juntos? ¿Cuál es el mayor sueldo? ¿Quién cobra más? ¿Cuál es la mayor edad? ¿Quién es más viejo? (Investigá para qué sirve `which.max()`).

8. Trabajá con la siguiente tabla de productos de un supermercado:

```
productos ← data.frame(
  nombre = c("Leche", "Pan", "Arroz", "Fideos", "Aceite", "Azucar",
    "Sal", "Harina", "Cafe", "Te", "Yerba", "Dulce de leche"),
  precio = c(450, 280, 320, 180, 890, 210, 95, 150, 1200, 380, 520, 410),
  stock = c(45, 120, 33, 88, 12, 67, 145, 55, 8, 22, 41, 29),
  categoria = c("Lacteos", "Panaderia", "Almacen", "Almacen", "Almacen",
    "Almacen", "Almacen", "Panaderia", "Infusiones",
    "Infusiones", "Infusiones", "Lacteos")
)
```

Respondé las siguientes preguntas operando vectorialmente:

- ¿Cuál es el producto más caro? ¿Y el más barato?
- ¿Cuál es el valor total del inventario?
- ¿Qué productos tienen stock menor a 30 unidades?
- ¿Cuál es el precio promedio de los productos de la categoría «Almacen» ?
- ¿Cuántos productos de «Infusiones» hay?
- Creá una columna **`valor_inventario`** que sea el producto del precio por el stock.
- Filtrá solo los productos lácteos que cuestan más de $400.
- ¿Qué productos son de la categoría «Almacen» Y tienen un stock mayor a 50 unidades?
- Encontrá los productos que cuestan menos de $300 O tienen un stock menor a 20 unidades.
- ¿Cuál es la categoría con mayor valor total de inventario?

9. Explorá el dataset `iris` (ya está cargado en R, así que solo llamalo por su nombre). ¿Qué tipos de información contiene? ¿Cuántas observaciones hay? Este es un dataset muy conocido. Cualquier duda, siempre se puede recurrir a internet.
10. En el dataset `iris`, ¿cuántas flores de la especie «setosa» tienen pétalos con ancho menor a 0.3?
11. Trabajando con `mtcars` (otro dataset famoso de R):
    - Echá un vistazo rápido a las primeras filas del dataset (`head()`).
    - ¿Qué variables (columnas) tiene disponibles? (`names()` o `colnames()`).
    - Guardá en una variable aparte solo los valores de rendimiento (`mpg`).
    - ¿Qué autos tienen más de 4 cilindros?
    - ¿Cuáles tienen más de 100 HP y son automáticos (`am == 0`)?
    - Identificá los autos «gastadores» : rinden menos de 15 mpg o pesan más de 4000 libras (`wt > 4`).
    - Convertí el rendimiento de millas/galón a km/litro (multiplicá `mpg` por 0.425) y agregalo como nueva columna.
    - ¿Cuánta potencia tiene en promedio un auto del dataset?
    - Los autos con 3 marchas suelen ser más viejos. ¿Cuál es el rendimiento promedio de esos autos?
    - Clasificá cada auto: es «potente» si tiene más de 150 HP, sino es un auto «normal».
    - Escribí código que imprima «Grande» si tiene más de 50 filas, sino «Pequeño».

12. Del dataset `empleados` anterior:
    - Ordená la tabla del más joven al más grande.
    - A Ana le dieron un aumento. Cambiá su sueldo a $1500.
    - Remové la columna de edades por políticas de privacidad.
    - Escribí código que recorra cada columna y, si es numérica, calcule su promedio.
13. Creá una tabla vacía que tenga dos columnas preparadas («A» y «B») pero sin datos.
14. Si sumás dos columnas de un data frame (`df$edad + df$sueldo`), ¿obtenés un solo número o un vector?
15. Tenés un data frame con valores faltantes (`NA`). Investigá cómo quedarte solo con las filas completas (`?na.omit()`).
16. Programá una cuenta regresiva del 10 al 1 que se imprima en consola.
17. Trabajá con el dataset `diamonds` (`library(ggplot2)`, pero antes averigüá como instalar el paquete 😉):
    - ¿Cuántos diamantes de corte «Ideal» cuestan más de $10000?
    - Convertí los quilates a gramos (1 quilate = 0.2 gramos) en una nueva columna.
    - Categorizá cada diamante como «Accesible» ($1000) o «Costoso» ($1000).
    - ¿Qué porcentaje de todos los diamantes son «Costosos» ?